

Failures of Supervised Learning Models in Practice: A Review of Data Leakage, Bias, Overfitting and Evaluation Pitfalls

Saatwik Sharma¹ and Dr. Priyanka Deshmukh¹

¹Symbiosis Institute of Technology,, Department of Artificial Intelligence and Machine Learning, Email: saatwik.sharma.btech205@sitpune.edu.in
priyanka.deshmukh@sitpune.edu.in

Abstract

Supervised learning models have achieved remarkable success across domains such as healthcare, finance and education. However, despite strong theoretical foundations and high validation performance during development, these models fail in real-world deployment. This review presents a deployment-centric analysis of why these models fail, with a focus on data leakage, dataset bias, overfitting, and evaluation pitfalls. We synthesize existing literature to examine how improper data handling—including train-test contamination, target leakage and preprocessing errors—can lead to overly optimistic results that do not generalize to real-world scenarios. This paper further analyzes the interaction between model capacity, distribution shift and evaluation design to highlight how these factors collectively undermine reliability in practice. Additionally, the review highlights dataset-related challenges including sampling bias, class imbalance and labelling errors, that significantly affect model performance. Overfitting and poor generalization are analysed beyond theoretical definitions and then further linked to domains like healthcare, finance and education. Finally, the paper discusses common evaluation mistakes and outlines best practices and mitigation strategies which improve the model at the time of real-world deployment.

Keywords: Supervised Learning, Data Leakage, Dataset Bias, Overfitting, Distribution Shift, Class Imbalance, Model Evaluation, Generalization, Validation Strategies.

1 Introduction

Machine Learning has rapidly become an essential component of modern data-driven systems, and plays a prominent role in domains like healthcare, education and finance. Most existing literature and educational resources focus heavily on model architecture, algorithms and performance metrics, often presenting machine learning as a process that ends once the model achieves high accuracy [1].

1.1 Motivation

Despite strong theoretical foundations and impressive validation metrics, many supervised learning models fail when deployed in real-world settings. Models that demonstrate

high accuracy during development often exhibit significant performance degradation in operational environments [2]. This inconsistency raises fundamental concerns regarding the reliability, robustness and generalizability of supervised learning systems. The growing dependence on automated decision systems in critical sectors makes understanding these failures not only a technical necessity but also a societal imperative.

While significant progress has been made in understanding how supervised learning models function mathematically and computationally, a deeper and more consequential question remains: will these models continue to function reliably under real-world conditions? Much of machine learning research focuses on algorithmic mechanisms, optimization techniques and benchmark performance, allowing researchers and practitioners to understand how a model learns from data. However, understanding how a model works is fundamentally different from ensuring that it will work when exposed to noisy, incomplete, shifting or operationally constrained environments [2].

In high-stakes domains such as healthcare diagnostics, financial risk assessment, and education evaluation, the practical reliability of a model is more important than its theoretical accuracy. This gap between conceptual understanding and real-world robustness motivates the need for a deployment-centric analysis of supervised systems [2].

1.2 Problem Statement

Modern machine learning systems are primarily developed under the framework of Empirical Risk Minimization (ERM), where models are optimized to minimize average loss on finite training datasets. While this paradigm provides strong mathematical foundations and has embarked remarkable performance improvements across benchmarks, it implicitly assumes that minimizing training loss leads to robust real-world performance. However, real-world environments are dynamic, uncertain and often structurally different from controlled training settings. As a result, models that demonstrate low empirical risk during development frequently exhibit performance degradation after deployment. This discrepancy raises a fundamental concern: optimization success on observed data does not guarantee reliability under operational conditions. Addressing the misalignment between theoretical optimization and practical robustness constitutes the central problem.

1.3 Scope of Review

This review focuses on the fundamental assumptions and practical limitations of supervised learning systems, particularly in relation to generalization and real-world deployment. The discussion centres on the theoretical framework of Empirical Risk Minimization (ERM) and its relationship to true risk, generalization error and the deployment-development gap [3].

The review examines critical factors that contribute to model failure beyond theoretical optimization, including violation of IID assumptions, distribution shift, dataset bias, data leakage, class imbalance, label noise and improper validation strategies. Emphasis is placed on understanding how these issues emerge within supervised learning pipelines and how they affect reliability after deployment [4].

Rather than proposing new algorithms, this review synthesizes existing theoretical principles and practical observations to analyse why models that perform well during development

fail under operational conditions. The scope is limited to supervised learning settings and does not extend to reinforcement learning, unsupervised learning or hardware-level optimization. The objective is to provide a structured examination of the disconnect between empirical optimization and real-world robustness [3, 4].

2 Foundation of Generalization and Deployment in Supervised Learning

2.1 Empirical Risk Minimization and Generalization Error

Empirical Risk Minimization is a learning principle in supervised learning in which the model parameters are chosen so as to minimize the average loss computed over the training dataset [5]. In supervised learning, the loss represents the difference between the actual target value and the model’s predicted output. For a training dataset containing multiple samples, a loss value is computed for every instance. The average of these individual losses across the entire training set is known as empirical risk [5]. The principle that guides the model to choose the parameters by minimizing the average training loss is referred to as Empirical Risk Minimization.

Mathematically, the empirical risk is:

$$R_{\text{emp}}(f) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) \quad (1)$$

This is called empirical risk. The model chooses the function that gives the lowest value. The reason behind it being called empirical is because it is calculated from the observed dataset and not from the true real-world distribution. It is based only on the sample which we have and not the real population. Empirical risk is sample-based error while true risk is population-based error [6]. This distinction is fundamental to understanding why models fail after deployment. The real goal is to minimize true risk—the actual objective is not to minimize the training loss but to minimize the true risk [6].

Mathematically, the true risk (expected risk) is:

$$R(f) = \mathbb{E}[L(Y, f(X))] \quad (2)$$

This represents the average prediction error over the entire real-world data distribution. Here X, Y are random variables. However, the true risk cannot be computed as the real-world distribution is unknown and we only observe a small sample [7]. The central problem causing models to fail is:

$$R_{\text{emp}}(f) \neq R_{\text{real}}(f) \quad (3)$$

The empirical risk is not equal to the true risk. Empirical Risk Minimization assumes that the probability distribution of the training data is identical to the distribution of the real world. The difference between them is called the *generalization gap* [8]. If the generalization gap is small, the model generalizes well; if the gap is too large, the model fails after deployment.

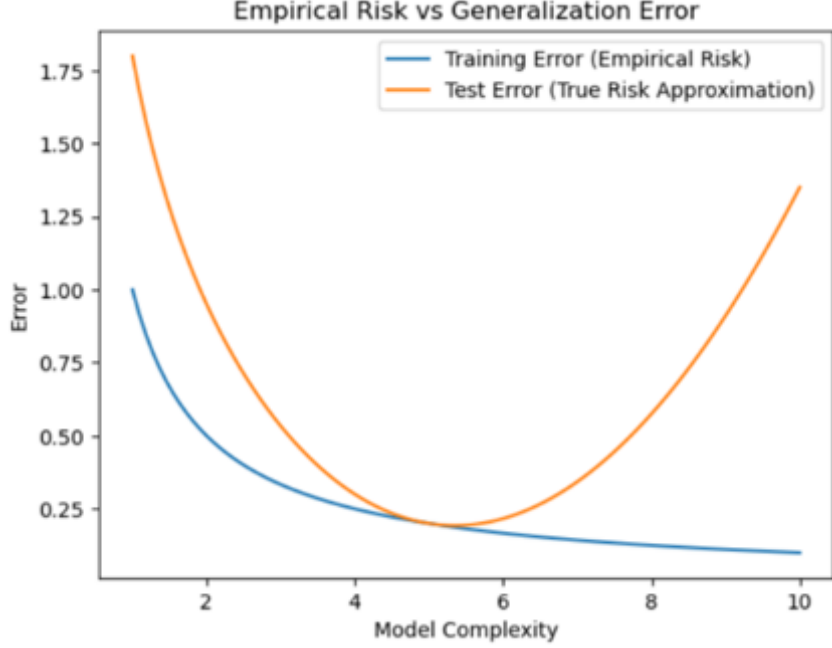


Figure 1: Relationship between model complexity, empirical risk and generalization error.

2.2 Assumptions of IID Data and Distribution Stability

Most machine learning theory, including ERM, assumes that data satisfies two major conditions: the IID assumption and distribution stability [9]. If these assumptions break, model performance collapses. IID stands for Independent and Identically Distributed, meaning two things: (1) *independence*—each data point does not depend on another; and (2) *identically distributed*—all samples are drawn from the same probability distribution, so that training and test data follow the same statistical pattern with no systematic difference between them [9].

Mathematically:

$$(X_i, Y_i) \sim P(X, Y) \quad \forall i \quad (4)$$

All samples come from the same underlying distribution $P(X, Y)$.

Distribution stability [10] means the probability distribution of data does not change over time. Formally:

$$P_{\text{train}}(X, Y) = P_{\text{deployment}}(X, Y) \quad (5)$$

ERM works under the IID assumption [11]: theoretical guarantees about generalization rely on IID, and if IID holds, empirical risk approximates the true risk. If IID fails, the empirical risk becomes misleading and the model fails in deployment. Dataset bias and data leakage are also reasons why the IID assumption fails.

2.3 The Development-Deployment Gap

The development-deployment gap refers to the difference between a model’s performance during development (training and validation phase) and its performance after real-world deployment [12]. In simple words, models perform well in the lab but fail in the real world. During development there are controlled datasets, clean preprocessing, balanced



Figure 2: Violation of IID assumption due to distribution shift between training and deployment data.

samples and a static environment. During deployment there is noisy data, changing distributions, missing values and operational constraints [12]. This mismatch creates performance degradation because the model’s evaluation during development does not fully replicate real-world conditions [13].

The key components that cause the gap are:

- **Evaluation Over-Optimism:** During development, cross-validation, hyperparameter tuning and multiple model comparisons can indirectly cause overfitting. This results in validation performance appearing high, with low deployment performance.
- **Hidden Data Leakage:** Leakage often goes unnoticed during development. For example, future information included accidentally or preprocessing done before splitting.
- **Infrastructure and Pipeline Differences:** In development there are clean pipelines, manual preprocessing and ideal computation, while in deployment there is real-world data and different software versions. Even small differences break model assumptions [14].
- **Concept Drift:** Over time, user behaviour changes and economic conditions shift; therefore the model becomes outdated. This widens the development-deployment gap.
- **Dataset Representatives:** Development datasets are often limited in size and geographically restricted, while the deployment population is wider. This causes fairness and reliability issues.

3 Data Leakage in Supervised Learning

3.1 Definition and Types of Data Leakage

Data leakage occurs when information is unintentionally used during model training or evaluation, leading to over-optimistic performance and unreliable behaviour. In supervised learning pipelines, leakage undermines the purpose of splitting training and testing data, as the model is learning from data that would not be present in real-world deployment [15]. In practice, leakage can occur at multiple stages of the machine learning workflow, including data preprocessing, feature engineering and even during evaluation; as a result, models are produced that appear highly accurate but fail in real-world deployment [15]. Common types of data leakage are [15]:

- **Train-Test Contamination:** This happens when the same or very similar data appears in both the training and testing datasets. It may happen when duplicate rows are not removed, resulting in overlapping data in training and testing sets.
- **Target Leakage:** This happens when the model is given information that already reveals the answer—essentially when a feature used for training directly encodes the target variable.
- **Temporal Leakage:** This happens when the model uses future information to predict the past. When data is randomly shuffled, future data appears in training [15].
- **Preprocessing Leakage:** This happens when a learning-based preprocessing step uses test data information during training. For example, scaling using mean and standard deviation calculated on the full dataset, or PCA computed on the entire dataset.

These types illustrate how leakage can arise not only from incorrect data partitioning but also from improper preprocessing and feature construction. All forms of leakage share the same hallmark: they allow the model to access information it should not have during training, misleading to high estimates [16].

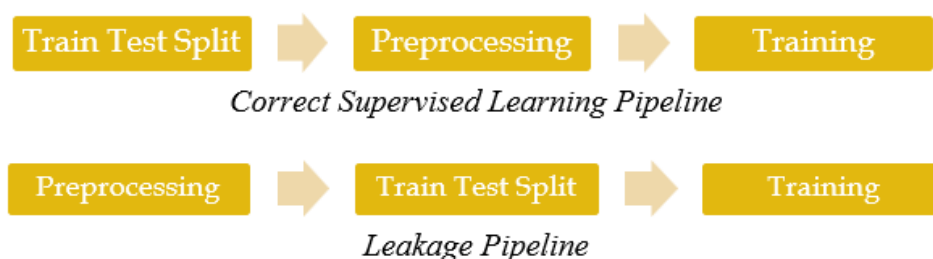


Figure 3: Comparison between correct supervised learning pipeline and leakage pipeline.

3.2 Common Sources of Leakage in Practice

Data leakage does not occur at a single step; it can occur at multiple steps in the supervised learning pipeline [17]. In practice, leakage is most commonly observed during feature engineering, preprocessing and model evaluation using cross-validation [17]. Although these stages serve different purposes, they share a common risk: allowing information from outside the training data to influence the learning process.

- **Leakage during Feature Engineering:** Feature engineering is the process of creating new input features from raw data. Leakage may occur when the engineered feature directly uses the target variable (target leakage). For example, to predict whether a student will pass or not, using the feature *Final exam score* leaks the information since the score only exists when the student has passed [17].
- **Leakage During Preprocessing:** There are two types of preprocessing: *safe (rule-based)* preprocessing done before data splitting, and *learning-based (statistical)* preprocessing that computes statistics such as scaling (mean and standard deviation), normalization, PCA and TF-IDF [17]. There is no risk of leakage in safe preprocessing, but in learning-based preprocessing leakage may occur when statistics are computed on the entire dataset before splitting into training and testing data.
- **Leakage During Cross-Validation:** Cross-validation itself does not cause leakage; it is only a method for splitting data. Leakage happens when preprocessing or feature selection is done before creating CV folds, causing each test fold to influence the training folds indirectly [18]. The correct approach is to split into folds first and then apply preprocessing within each fold.

Machine learning pipelines are recommended because they ensure that all steps of the learning process are executed in a fixed, consistent and reproducible order [18]. Without pipelines, models may perform well during development but fail during real-world deployment due to inconsistencies in preprocessing, feature engineering or evaluation [19]. Pipelines ensure the same scaler, the same encoder and the same feature transformations are automatically applied to new data. Without pipelines, feature selection may use the full dataset and scaling may include validation-fold data; pipelines automatically fit preprocessing inside each fold and keep validation data unseen during training [19].

4 Dataset Bias and Labelling Issues

4.1 Sampling and Representation Bias

Sampling and representation bias are two closely related types of dataset bias that frequently arise in supervised machine learning. Sampling bias occurs when the method used to collect or select data points results in a dataset that is not representative of the underlying population [20]. For instance, in facial recognition datasets, images are often collected from social media platforms, leading to an overrepresentation of younger individuals and light-skinned faces. As a result, models trained on such datasets perform well on the majority group but fail to generalize to underrepresented populations. Sampling bias focuses on the process of collection, while representation bias [21] focuses on the composition of datasets—it is an imbalance in the dataset’s coverage and diversity rather than solely in the method of sampling [20]. For example, a medical imaging dataset used for disease diagnosis may have predominantly male patients, resulting in poor model performance on female patients.

These biases are particularly problematic in domains like healthcare and autonomous systems where model errors can have severe consequences. Therefore, the composition of datasets must be considered in order to develop reliable and generalizable machine learning models [20, 21].

4.2 Class Imbalance in Supervised Datasets

Class imbalance refers to a common characteristic in supervised datasets in which the distribution of class labels is highly uneven, meaning one class (the *majority class*) contains more samples and another (the *minority class*) contains fewer samples [22]. In supervised learning, if class imbalance exists, the learning algorithm receives more information about the majority class during training; therefore the model learns mostly from the majority class, effectively ignoring the minority class [23].

A widely cited example of class imbalance is fraud detection. In datasets containing credit card transactions, more than 99% of transactions are legitimate while less than 1% are fraudulent. A classifier trained on such a dataset may achieve over 99% accuracy by simply predicting every transaction as legitimate. However, despite this high accuracy, the model completely fails at detecting fraud, rendering it ineffective for real-world deployment [23].

4.3 Label Noise and Annotation Errors

Label noise and annotation errors are the inconsistencies that arise in a dataset due to human mistakes, subjective interpretations, or errors in the annotation process [24]. Since the model clearly relies on the correct answer (the label), these errors may significantly degrade model performance. Label noise is very common in large-scale datasets; it happens when a wrong label is given to data by mistake—for example, an image of a cat labelled as a dog, or a medical X-ray showing disease labelled as healthy [24]. This reduces accuracy and leads to poor generalization to unseen data.

Annotation error happens when an annotator makes a mistake while labelling data. These errors occur due to human mistakes generally [25]. For example, two doctors may label the same medical image differently. Annotation errors are especially harmful in high-risk applications such as healthcare, autonomous driving and security systems where incorrect predictions can lead to serious consequences. To address these challenges, it is important to carefully curate datasets, involve expert validation, check consistency between annotators and use techniques that are robust to noisy data.

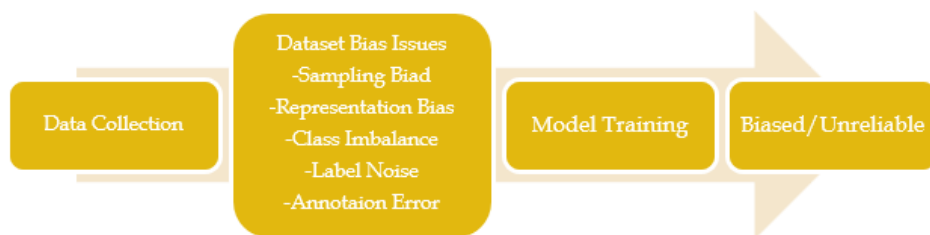


Figure 4: Common sources of dataset bias and labelling issues.

5 Overfitting and Generalization Failures

5.1 Understanding Overfitting Beyond Theory

Overfitting is commonly defined as a situation where a model performs very well on training data but poorly on unseen data [26]. While theoretical explanations often

describe overfitting as a result of excessive model complexity relative to dataset size, in real-world scenarios it is more complex and multi-dimensional.

In practice, overfitting emerges from the interaction between:

- Model capacity
- Dataset size and quality
- Noise and label errors
- Feature engineering choices
- Data leakage
- Evaluation strategy

A high-capacity model (such as a deep neural network or ensemble method) can approximate complex functions. However, if the dataset contains noise, outliers or miscorrelations, the model may memorize these patterns instead of learning true underlying structures, leading to memorization rather than generalization. Importantly, overfitting is not limited to small datasets. Even large-scale models trained on massive datasets can overfit if the training distribution does not match the deployment distribution [27]. In such cases, the model appears robust during validation but fails in real-world settings. Another practical dimension of overfitting is feature engineering, because excessive feature selection and repeated hyperparameter tuning on the same validation set can cause validation-specific patterns, leading to what is often called *validation set overfitting*.

Hence, overfitting should not be merely seen as a theoretical bias issue, but as a systemic pipeline failure involving data collection, preprocessing, model selection and evaluation design [26, 27].

5.2 Bias-Variance Trade-off

The bias-variance trade-off is a fundamental concept used to explain why machine learning models sometimes fail to generalize well on new data. *Bias* refers to the error caused by overly simplistic assumptions in the learning algorithm. When a model is too simple to capture the underlying relationship in the data, it makes systematic errors, a condition known as *underfitting*. A high-bias model oversimplifies the problem and misses important patterns.

Variance refers to the model’s sensitivity to small changes in the training dataset. A high-variance model learns the training data too closely, including noise and random fluctuations—this is called *overfitting* [26]. A high-variance model performs well on training data but poorly on testing data.

Bias and variance usually move in opposite directions: for simple models there is high bias and low variance, while for complex models there is low bias and high variance [28]. If model complexity is increased, bias decreases and variance increases; if model complexity is decreased, variance decreases and bias increases. The main goal is to find an optimal balance where total prediction error is minimized. In practical terms, good generalization occurs when the model captures true patterns (low bias) and ignores random noise (controlled variance) [28].

6 Evaluation Pitfalls in Machine Learning

6.1 Limitation of Accuracy as a Metric

In supervised learning, accuracy is one of the most commonly used evaluation metrics and is defined as:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}} \quad (6)$$

Because of its simplicity, accuracy is often used as the first metric reported in research papers and machine learning projects. However, despite its popularity, accuracy can be misleading and prove ineffective for real-world applications. Accuracy treats all prediction errors equally and does not distinguish between false positives and false negatives [29]. For example, in medical diagnosis, predicting a healthy person as sick (false positive) and predicting a sick person as healthy (false negative) are both counted as wrong, yet the second is far more dangerous; accuracy does not distinguish between these errors [29].

Another major limitation arises with imbalanced datasets. For example, in email spam detection where 95% of emails are not spam and 5% are spam, if a model predicts all emails as not spam then the accuracy is 95%—a seemingly good number. However, it fails to detect any spam emails. The model is practically useless yet accuracy suggests high performance; this is called the *accuracy paradox* [30].

To understand why accuracy is limited, consider the confusion matrix shown in Table 1.

Table 1: Confusion matrix for binary classification.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Accuracy can be calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

Note that accuracy combines predictions from both classes and does not separately measure performance on the minority class. Thus, a model may have high TN and very low TP and yet still achieve high accuracy. Accuracy provides an aggregate measure of correctness but fails to capture the distribution and contextual impact of classification errors [30]. Despite its limitations, accuracy is not useless: it works well when the data is balanced, the costs of false positives and false negatives are similar, and the problem is a low-risk classification. Alternatives to accuracy include precision, recall and F1-score [30].

6.2 Misuse of Training and Test Sets

In supervised learning, data is typically divided into two sets: training and testing. The fundamental principle of evaluation is that the model must be tested on data it has never seen before. If this principle is violated, performance estimates become unreliable and overly optimistic. In practice, training and test sets are often misused, leading to inflated accuracy and deployment failures. Misuse happens when information from the test set

influences the training process, test data is repeatedly used for model tuning, or data splitting is done incorrectly [31]. These mistakes lead to data leakage. Correct practice is to split the data first and then apply normalization. Another mistake is repeated use of the test set for tuning; after training and evaluating on the test set, modifying hyperparameters and evaluating on the same test set effectively converts the test set into a validation set—a phenomenon called *test set contamination*. Improper data splitting strategies and duplicate or related samples across sets are also very common [31].

6.3 Improper Validation Strategies

Validation answers the question: if model settings are changed, which version works better on unseen data? Validation is used for hyperparameter tuning, model selection, early stopping and architecture comparison. If validation is wrong, model selection is wrong and therefore deployment fails. As discussed, empirical risk is training error, true risk is real-world error, and the generalization gap is the difference between them. Validation error is used as an estimate of true risk [32]. Improper validation means the validation process does not realistically simulate unseen real-world data [33]. It creates over-optimistic performance estimates. Major types of improper validations [33] include:

- Validation set overuse (Repeated Tuning)
- Data leakage during Validation
- Wrong Cross-Validation
- Random validation in Time-series problems
- Non-representative validation data
- Single Metric Validation

Mathematically, let $R_{\text{val}}(f)$ be the validation risk. We assume that:

$$R_{\text{val}}(f) \approx R_{\text{real}}(f) \tag{8}$$

But with improper validation:

$$R_{\text{val}}(f) < R_{\text{real}}(f) \tag{9}$$

So validation underestimates real error. That is the core issue for the development-deployment gap. Validation is not just checking performance; it is a simulation of deployment—if the simulation is unrealistic, deployment will fail [33].

7 Case-Based Observation in Real-World Applications

7.1 Healthcare Decision Support Systems

Healthcare is a high-stakes domain where small prediction errors can directly affect patient safety and survival, making model reliability critically important. Consider a scenario where a hospital builds a machine learning model to predict whether a patient will have a heart attack in the next 48 hours [34,35]. The training data comes from one private

urban hospital consisting mostly of middle-class patients aged 40–65 with the same lab machines and diagnostic protocols. The model assumes that the training distribution and the real-world data distribution are equal (IID assumption). When deployed to a different distribution, the assumption breaks and the model starts making wrong predictions.

The dataset has 70% male patients and only 30% female patients. Heart attack symptoms in females are often different, while the model learns only male patterns—after deployment it misses heart attack signs in women. Because the dataset was not representative, this is called *representative bias*. Furthermore, 95% of the patients do not get a heart attack while 5% do; so the model learns to predict ‘No heart attack’ and achieves 95% accuracy. But in reality, it fails to detect actual heart attack cases and the accuracy is misleading [35].

Data leakage is very common in healthcare. During preprocessing, blood pressure and cholesterol values are normalized using the entire dataset before splitting into training and validation, causing validation statistics to leak into training. One feature used was “Troponin test result taken 6 hours after admission”—but this test is often ordered after the doctor already suspects a heart attack. The model indirectly accesses future information: this is called *target leakage*. Heart attack labels were based on discharge summaries, but some doctors labelled the condition as ‘cardiac arrest’ and others as ‘acute coronary syndrome’—the same condition with different labels, confusing the model and causing it to learn noisy patterns. This is annotation inconsistency.

Model overfitting also occurs: the validation accuracy is 93% but the deployment accuracy is 74%, because the model memorized hospital-specific patterns rather than universal medical patterns. Evaluation metrics such as recall for heart attack cases, false negative rate, precision and calibration were never checked. In healthcare, missing a heart attack (false negative) is far worse than a false alarm, but accuracy treats all errors equally.

Table 2: Real-world dataset of prediction of heart attack (self-collected).

Patient ID	Age	Blood Pressure	Troponin Level	Diagnosis Label
P001	58	180	0.8	Heart attack
P002	62	175	0.7	No Heart attack
P003	45	120	0.1	Heart attack
P004	59	NULL	0.6	Cardiac arrest
P005	61	182	0.5	Heart attack

Table 2 illustrates how real-world healthcare datasets inherently contain structural imperfections, including label inconsistency, missing values, suspicious labels, target leakage, class imbalance and distribution shift. After deployment, the model missed 18% of real heart attack cases; doctors lost trust and the system was temporarily disabled. Investigation revealed validation bias and data shift. Healthcare is especially sensitive because data is heterogeneous, labels are subjective and the population varies—small mistakes can cost lives [35].

7.2 Financial Risk and Fraud Detection

The financial domain is one of the most critical and high-impact application areas of machine learning [36, 37]. Financial institutions use machine learning models to detect fraudulent transactions, assess credit risk and prevent financial losses. However, unlike

textbook datasets, financial data is highly dynamic, imbalanced and adversarial in nature, making model reliability a serious challenge.

Fraud data in a dataset is extremely rare: 99.7% of transactions are legitimate while only 0.3% are fraudulent. A model can achieve 99% accuracy by predicting “not fraud” every time, yet completely fail at detecting fraud. In finance, customer behaviour changes and fraud tactics evolve, so the IID assumption does not hold true. This change of environment behaviour causes the model to fail in real-world deployment—also called *concept drift*. If the features include “chargeback status”, the model may accidentally learn future information, resulting in unrealistically high validation performance. Training data often contains more urban customers, specific income groups and historical lending patterns, creating bias against certain demographics and resulting in unfair credit scoring models [36].

	Predicted Fraud	Predicted Legit
Actual Fraud	45	55
Actual Legit	20	9880

As financial environments are highly dynamic, the statistical properties of the data change over time. These changes are commonly referred to as *data drift* and *concept drift*, and both can significantly degrade model performance. Data drift occurs when the distribution of input features changes over time, even though the relationship between input and output remains the same. Concept drift occurs when the relationship between input features and target variables changes over time. For example, fraudsters constantly change their strategies—previously, large international transactions were suspicious, but now fraudsters make many small transactions to avoid detection. The meaning of “fraud” given certain transaction patterns changes, causing the model to learn outdated fraud patterns and fraud cases to go undetected [36, 37].

Adversarial behaviour is a significant challenge in financial machine learning systems because fraud detection models operate in an environment where malicious actors actively attempt to bypass automated decision systems. Fraudsters often study patterns in transaction monitoring systems and modify their behaviours to avoid detection. These manipulations may include splitting a large fraudulent transaction into several smaller transactions, slightly changing transaction timing or location, or mimicking normal customer spending patterns. Although these changes may appear insignificant from a data perspective, they can shift feature values enough to avoid triggering fraud detection rules learned during training, leading to performance degradation and potential financial loss [37].

8 Mitigation Strategies and Best Practices

8.1 Data-Centric Approaches

There are two ways to improve a machine learning system. The first is the *model-centric approach*, where the main goal is to improve the algorithm. The second is the *data-centric approach*, where the main goal is to improve data instead of the model [38]. The idea is that better data automatically makes the model better. Major data-centric strategies include [38]:

- **Strategy 1: Dataset Auditing.** Carefully analyse the dataset before training. Check for duplicates, missing values, label errors or imbalance using exploratory data analysis, distribution analysis or statistical analysis [39]. The goal is to understand what is wrong with the dataset.
- **Strategy 2: Data Cleaning.** Remove bad data by removing duplicates, correcting labels, handling missing values, normalizing formats and removing outliers [39]. For example, a transaction dataset may contain \$100, 100, 100.00—all these values must be standardized.
- **Strategy 3: Balanced Sampling.** Fix class imbalance using oversampling (increasing minority class), undersampling (reducing majority class) or synthetic data generation. A common technique is SMOTE (Synthetic Minority Oversampling Technique).
- **Strategy 4: Better Data Collection.** Instead of manipulating data artificially, collect better data. For a healthcare dataset, data should include multiple hospitals, multiple regions, different age groups and both genders to improve representativeness [38].
- **Strategy 5: Data Augmentation.** Create new training examples from existing data. Common in computer vision (rotate, flip, crop images, adjust brightness) and text data (synonym replacement, paraphrasing, back translation). The goal is to increase dataset diversity.
- **Strategy 6: Continuous Dataset Updates.** The real world changes over time; therefore datasets must be updated periodically. For example, a fraud detection model should be retrained every few weeks because fraud patterns change.

Data-centric approaches improve machine learning reliability by improving dataset quality, representativeness, labelling accuracy and distribution coverage rather than only modifying model architectures [38].

8.2 Robust Evaluation Techniques

The main goal of robust evaluation is to ensure that model performance measured during development is close to real-world deployment. Many models fail not because the model is weak but because evaluation methods are weak or unrealistic. Below are some robust evaluation techniques [40, 41].

- **Proper Train-Validation-Test Split.** The dataset should be divided into three parts: (1) Training Set (model learns patterns), (2) Validation Set (hyperparameter tuning), and (3) Test Set (final unbiased evaluation). The important rule is that the test set must remain completely unseen during training and model tuning. Developers should use the test set only once for the final evaluation [40].
- **Cross-Validation.** Cross-validation is used when datasets are small or limited. Instead of using a single split, the dataset is divided into multiple folds [40]. In K-Fold Cross-Validation: split the dataset into K equal parts, train on $K - 1$ folds, validate on the remaining fold, repeat K times, and average the performance. This produces a stable, more reliable performance estimate.
- **Stratified Sampling.** When a dataset is imbalanced, random splitting may produce uneven class distributions. Stratified sampling ensures that the training, validation,

and test sets all contain the same class proportions, ensuring fair evaluation.

- **Time-Aware Validation (For Time-Series Data).** For temporal data such as finance, stock prediction, fraud detection and user behaviour, random shuffling is incorrect because it introduces temporal leakage. The correct approach is chronological splitting, for example: Training data (2018–2023), Validation data (2024), Test data (2025).
- **Multiple Evaluation Metrics.** As discussed, accuracy alone is misleading. Robust evaluation requires multiple metrics including Precision, Recall, F1-score, ROC-AUC and Calibration.
- **Out-of-Distribution Testing.** Models should also be evaluated on data slightly different from the training data. For example, a healthcare model trained on one hospital should also be tested on other hospitals, different demographic groups, or different equipment. This reveals whether the model truly generalizes [41].

9 Conclusion

Supervised learning models have achieved significant success across domains such as healthcare, finance and education. However, as this review has highlighted, strong validation performance during development does not necessarily guarantee reliable performance in real-world deployment [42]. Many failures arise from issues embedded within the machine learning pipeline, including data leakage, dataset bias, class imbalance, label noise and improper validation strategies. These factors produce overly optimistic evaluation results while masking deeper problems related to generalization and distribution shift.

This review emphasizes that improving the reliability of supervised learning systems requires a broader perspective that extends beyond model architecture. Representative datasets, preventing leakage during preprocessing, and adopting robust evaluation [43] techniques are essential for narrowing the development-deployment gap. By prioritizing data-centric approaches and rigorous validation techniques, researchers can develop machine learning systems that are more robust, trustworthy and suitable for the real world.

References

- [1] Zhou, Z. H. (2021). *Machine Learning*. Springer Nature.
- [2] Naser, M. Z. (2025). From failure to fusion: A survey on learning from bad machine learning models. *Information Fusion*, 120, 103122.
- [3] Niveditha, H. R., Balakrishna, K., & Anitha, S. (2024). With future scope. *Proceedings of the First Artificial Intelligence Summit on Smart Sustainable Society: AISSS 2023*, 125.
- [4] Zub, S. (2019, April). A(n) assumption in machine learning. In *COLINS 2019. Volume II: Workshop*.

- [5] Donini, M., Oneto, L., Ben-David, S., Shawe-Taylor, J. S., & Pontil, M. (2018). Empirical risk minimization under fairness constraints. *Advances in Neural Information Processing Systems*, 31.
- [6] Li, T., Beirami, A., Sanjabi, M., & Smith, V. (2020). Tilted empirical risk minimization. *arXiv preprint arXiv:2007.01162*.
- [7] Montanari, A., & Saeed, B. N. (2022, June). Universality of empirical risk minimization. In *Conference on Learning Theory* (pp. 4310–4312). PMLR.
- [8] Nagarajan, V., Andreassen, A., & Neyshabur, B. (2020). Understanding the failure modes of out-of-distribution generalization. *arXiv preprint arXiv:2010.15775*.
- [9] Inlow, M. (2006, June). Teaching the IID assumption in engineering statistics I. In *2006 Annual Conference & Exposition* (pp. 11–1230).
- [10] Ostapenko, O., Rodríguez, P., Lacoste, A., & Charlin, L. (2023, June). From IID to the independent mechanisms assumption in continual learning. In *AAAI Bridge Program on Continual Causality* (pp. 25–29). PMLR.
- [11] Garg, S. (2024). *Robust Machine Learning: Detection, Evaluation and Adaptation under Distribution Shift* (Doctoral dissertation, Carnegie Mellon University, USA).
- [12] Honkanen, T., O’Dwyer, J., Salminen, V., & Jussila, J. (2024, March). Development and deployment challenges of machine learning systems. In *International Conference on Information Management* (pp. 180–193). Cham: Springer Nature Switzerland.
- [13] Paleyes, A., Urma, R. G., & Lawrence, N. D. (2022). Challenges in deploying machine learning: a survey of case studies. *ACM Computing Surveys*, 55(6), 1–29.
- [14] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., . . . , & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28.
- [15] Apicella, A., Isgrò, F., & Prevete, R. (2025). Don’t push the button! Exploring data leakage risks in machine learning and transfer learning. *Artificial Intelligence Review*, 58(11), 339.
- [16] Bouke, M. A., Zaid, S. A., & Abdullah, A. (2024). Implications of data leakage in machine learning preprocessing: a multi-domain investigation.
- [17] Sasse, L., Nicolaisen-Sobesky, E., Dukart, J., Eickhoff, S. B., Götz, M., Hamdan, S., . . . , & Patil, K. R. (2025). Overview of leakage scenarios in supervised machine learning. *Journal of Big Data*, 12(1), 135.
- [18] Sasse, L., Nicolaisen-Sobesky, E., Dukart, J., Eickhoff, S. B., Götz, M., Hamdan, S., . . . , & Patil, K. R. (2023). On leakage in machine learning pipelines. *arXiv preprint arXiv:2311.04179*.
- [19] Posoldova, A. (2020). Machine learning pipelines: From research to production. *IEEE Potentials*, 39(6), 38–42.
- [20] Panzeri, S., Magri, C., & Carraro, L. (2008). Sampling bias. *Scholarpedia*, 3(9), 4258.

- [21] Shahbazi, N., Lin, Y., Asudeh, A., & Jagadish, H. V. (2023). Representation bias in data: A survey on identification and resolution techniques. *ACM Computing Surveys*, 55(13s), 1–39.
- [22] Narwane, S. V., & Sawarkar, S. D. (2019). Machine learning and class imbalance: A literature survey. *Industrial Engineering Journal*, 12(10.26488).
- [23] Guo, X., Yin, Y., Dong, C., Yang, G., & Zhou, G. (2008, October). On the class imbalance problem. In *2008 Fourth International Conference on Natural Computation* (Vol. 3, pp. 192–201). IEEE Computer Society.
- [24] Beigman, E., & Klebanov, B. B. (2009, August). Learning with annotation noise. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP* (pp. 280–287).
- [25] Johnson, J. M., & Khoshgoftaar, T. M. (2022). A survey on classifying big data with label noise. *ACM Journal of Data and Information Quality*, 14(4), 1–43.
- [26] Dietterich, T. (1995). Overfitting and undercomputing in machine learning. *ACM Computing Surveys (CSUR)*, 27(3), 326–327.
- [27] Ying, X. (2019, February). An overview of overfitting and its solutions. In *Journal of Physics: Conference Series* (Vol. 1168, No. 2, p. 022022). IOP Publishing.
- [28] Briscoe, E., & Feldman, J. (2011). Conceptual complexity and the bias/variance tradeoff. *Cognition*, 118(1), 2–16.
- [29] Dumas, B. T. (1997). The evolution and limitations of accuracy and precision standards. *Clinica Chimica Acta*, 260(2), 145–162.
- [30] Foody, G. M. (2023). Challenges in the real world use of classification accuracy metrics: From recall and precision to the Matthews correlation coefficient. *PLOS ONE*, 18(10), e0291908.
- [31] Kapoor, S., & Narayanan, A. (2022). Leakage and the reproducibility crisis in ML-based science. *arXiv preprint arXiv:2207.07048*.
- [32] Burzykowski, T., Geubbelmans, M., Rousseau, A. J., & Valkenborg, D. (2023). Validation of machine learning algorithms. *American Journal of Orthodontics and Dentofacial Orthopedics*, 164(2), 295–297.
- [33] Lopez, E., Gorla, G., Etxebarria-Elezgarai, J., Amigo, J. M., & Seifert, A. (2025). The importance of choosing a proper validation strategy in predictive models. Part 2: Recipes for (avoiding) overfitting. *Analytica Chimica Acta*, 344838.
- [34] Aliferis, C., & Simon, G. (2024). Lessons learned from historical failures, limitations and successes of AI/ML in healthcare and the health sciences: Enduring problems, and the role of best practices. *Artificial Intelligence and Machine Learning in Health Care and Medical Sciences: Best Practices and Pitfalls*, 543–606.
- [35] Garg, A., Venkataramani, V. V., Karthikeyan, A., & Priyakumar, U. D. (2022, January). Modern AI/ML methods for healthcare: Opportunities and challenges. In *International Conference on Distributed Computing and Internet Technology* (pp. 3–25). Cham: Springer International Publishing.

- [36] Singh, K., Hasan, M., & Rajendran, R. P. (2023). Opportunities and challenges of AI/ML in finance. *The Impact of AI Innovation on Financial Sectors in the Era of Industry 5.0*, 238–260.
- [37] Cao, L. (2022). AI in finance: Challenges, techniques, and opportunities. *ACM Computing Surveys (CSUR)*, 55(3), 1–38.
- [38] Singh, P. (2023). Systematic review of data-centric approaches in artificial intelligence and machine learning. *Data Science and Management*, 6(3), 144–157.
- [39] Camizuli, E., & Carranza, E. J. (2018). Exploratory data analysis (EDA). *The Encyclopedia of Archaeological Sciences*, 1–7.
- [40] Brown, O., Curtis, A., & Goodwin, J. (2021). Principles for evaluation of AI/ML model performance and robustness. *arXiv preprint arXiv:2107.02868*.
- [41] Zhou, J., Gandomi, A. H., Chen, F., & Holzinger, A. (2021). Evaluating the quality of machine learning explanations: A survey on methods and metrics. *Electronics*, 10(5), 593.
- [42] Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., & Aroyo, L. M. (2021, May). “Everyone wants to do the model work, not the data work”: Data cascades in high-stakes AI. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (pp. 1–15).
- [43] Recht, B., Roelofs, R., Schmidt, L., & Shankar, V. (2019, May). Do ImageNet classifiers generalize to ImageNet? In *International Conference on Machine Learning* (pp. 5389–5400). PMLR.